

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки (специальность) **09.03.03 Прикладная информатика**

ПРАКТИЧЕСКАЯ РАБОТА № 2

По дисциплине «Алгоритмы и структуры данных»

Тема: Двоичные деревья поиска.
Вариант 1.

Выполнила	Борзова Д. М.; К3140.
Проверила	Ромакина О. М.
Дата	_____

Санкт-Петербург 2026

Оглавление

Оглавление	2
Задачи по варианту	4
1 Задача. Обход двоичного дерева [5 s, 512 Mb]	4
Описание задания к задаче	4
Исходный код и описание решения	4
Описание проведённых тестов	6
Вывод по задаче	9
12 Задача. Проверка сбалансированности [2 s, 256 Mb]	10
Описание задания к задаче	10
Исходный код и описание решения	10
Описание проведённых тестов	12
Вывод по задаче	13
17 Задача. Множество с суммой [120 s, 512 Mb]	14
Описание задания к задаче	14
Исходный код и описание решения	15
Описание проведённых тестов	20
Вывод по задаче	24
Дополнительные задачи	25
3 Задача. Простейшее BST [2 s, 256 Mb, 1 балл, с учётом штрафа 0,5 балла]	25
Описание задания к задаче	25
Исходный код и описание решения	25
Описание проведённых тестов	27
Вывод по задаче	30
5 Задача. Простое двоичное дерево поиска [2 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]	31
Описание задания к задаче	31
Исходный код и описание решения	31
Описание проведённых тестов	35
Вывод по задаче	36
7 Задача. Опознавание двоичного дерева поиска (усложненная версия) [10 s, 512 Mb, 2,5 балла, с учётом штрафа 2 балла]	37
Описание задания к задаче	37
Исходный код и описание решения	38
Описание проведённых тестов	39
Вывод по задаче	43

8 Задача. Высота дерева возвращается [2 s, 256 Mb, 2 балла, с учётом штрафа 1,5 балла]	44
Описание задания к задаче	44
Исходный код и описание решения	44
Описание проведённых тестов	46
Вывод по задаче	47
10 Задача. Проверка корректности [2 s, 256 Mb, 2 балла, с учётом штрафа 1,5 балла] ..	48
Описание задания к задаче	48
Исходный код и описание решения	48
Описание проведённых тестов	50
Вывод по задаче	52
11 Задача. Сбалансированное двоичное дерево поиска [2 s, 512 Mb, 2 балла, с учётом штрафа 1,5 балла]	53
Описание задания к задаче	53
Исходный код и описание решения	53
Описание проведённых тестов	57
Вывод по задаче	59
16 Задача. К-й максимум [2 s, 512 Mb, 3 балла, с учётом штрафа 2,5 балла]	60
Описание задания к задаче	60
Исходный код и описание решения	60
Описание проведённых тестов	64
Вывод по задаче	66
Вывод по проделанной работе	67

Задачи по варианту

1 Задача. Обход двоичного дерева [5 s, 512 Mb]

Описание задания к задаче

В этой задаче вы реализуете три основных способа обхода двоичного дерева «в глубину»: центрированный (in-order), прямой (pre-order) и обратный (post-order).

Вам дано корневое двоичное дерево. Выведите центрированный (in-order), прямой (pre-order) и обратный (post-order) обходы в глубину.

- **Формат ввода: стандартный ввод или input.txt.** В первой строке входного файла содержится количество узлов n . Узлы дерева пронумерованы от 0 до $n - 1$. Узел 0 является корнем.
Следующие n строк содержат информацию об узлах 0, 1, ..., $n - 1$ по порядку. Каждая из этих строк содержит три целых числа K_i , L_i и R_i . K_i – ключ i -го узла, L_i – индекс левого ребёнка i -го узла, а R_i – индекс правого ребёнка i -го узла. Если у i -го узла нет левого или правого ребёнка (или обоих), соответствующие числа L_i или R_i (или оба) будут равны -1 .
- **Ограничения на входные данные.** $1 \leq n \leq 10^5$, $0 \leq K_i \leq 10^9$, $-1 \leq L_i, R_i \leq n - 1$. Гарантируется, что данное дерево является двоичным деревом. В частности, если $L_i \neq -1$ и $R_i \neq -1$, то $L_i \neq R_i$. Кроме того, узел не может быть ребёнком двух разных узлов. Кроме того, каждый узел является потомком корневого узла.
- **Формат вывода / выходного файла (output.txt).** Выведите три строки. Первая строка должна содержать ключи узлов при центрированном обходе дерева (in-order). Вторая строка должна содержать ключи узлов при прямом обходе дерева (pre-order). Третья строка должна содержать ключи узлов при обратном обходе дерева (post-order).
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
def main():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    n = int(lines[0].strip())
    keys = [0] * n
```

```

left = [0] * n
right = [0] * n

for i in range(n):
    parts = lines[i + 1].strip().split()
    keys[i] = int(parts[0])
    left[i] = int(parts[1])
    right[i] = int(parts[2])

in_order = []
stack = []
current = 0
while stack or current != -1:
    if current != -1:
        stack.append(current)
        current = left[current]
    else:
        current = stack.pop()
        in_order.append(str(keys[current]))
        current = right[current]

pre_order = []
stack = [0]
while stack:
    current = stack.pop()
    pre_order.append(str(keys[current]))
    if right[current] != -1:
        stack.append(right[current])
    if left[current] != -1:
        stack.append(left[current])

post_order = []
stack = [(0, False)]
while stack:
    current, processed = stack.pop()
    if processed:
        post_order.append(str(keys[current]))
    else:
        stack.append((current, True))
        if right[current] != -1:
            stack.append((right[current], False))
        if left[current] != -1:
            stack.append((left[current], False))

with open('output.txt', 'w') as f:
    f.write(' '.join(in_order) + '\n')
    f.write(' '.join(pre_order) + '\n')
    f.write(' '.join(post_order) + '\n')

if __name__ == '__main__':
    main()

```

Читаем из input.txt количество вершин n , затем n строк с описанием каждой вершины: ключ K_i , индекс левого L_i и правого R_i ребёнка. Храним дерево в трёх параллельных массивах. Реализуем три обхода в глубину итеративно, с использованием явного стека -

это принципиально важно, так как дерево может быть глубиной до 10^5 и рекурсия вызовет переполнение стека вызовов.

Для центрированного обхода (in-order) идём по левым детям, складывая путь в стек; дойдя до -1, извлекаем вершину, записываем её ключ и переходим к правому ребёнку.

Для прямого обхода (pre-order) помещаем корень в стек; при извлечении записываем ключ и кладём сначала правого, потом левого ребёнка, чтобы левый обработался первым. Для обратного обхода (post-order) используем стек с флагом «обработаны ли дети»: при первом посещении кладём вершину обратно с флагом True и добавляем детей, при втором - записываем ключ. Каждый из трёх обходов сохраняет ключи в отдельный список, после чего все три строки записываются в output.txt.

Сложность - $O(n)$ по времени и $O(n)$ по памяти для хранения дерева и стеков.

Описание проведённых тестов

input.txt			
1	5		
2	4 1 2		
3	2 3 4		
4	5 -1 -1	1	1 2 3 4 5
5	1 -1 -1	2	4 2 1 3 5
6	3 -1 -1	3	1 3 2 5 4
7		4	

```

input.txt x
1      10
2      0 7 2
3      10 -1 -1
4      20 -1 6
5      30 8 9
6      40 3 -1
7      50 -1 -1
8      60 1 -1
9      70 5 4
10     80 -1 -1
11     90 -1 -1
12

```

```

output.txt x
1      50 70 80 30 90 40 0 20 10 60
2      0 70 50 40 30 80 90 20 60 10
3      50 80 90 30 40 70 10 60 20 0
4

```

```

6 ▶ def test_minimal():  Ⓜ MarieeMorte
7     check( task_num: 1, input_data: "1\n42 -1 -1\n", expected: "42\n42\n42")
8
9
10 ▶ def test_example_1():  Ⓜ MarieeMorte
11     check(
12         task_num: 1,
13         input_data: "5\n"
14         "4 1 2\n"
15         "2 3 4\n"
16         "5 -1 -1\n"
17         "1 -1 -1\n"
18         "3 -1 -1\n",
19         expected: "1 2 3 4 5\n"
20         "4 2 1 3 5\n"
21         "1 3 2 5 4",
22     )

```

```

25  def test_example_2():  MarieeMorte
26      check(
27          task_num: 1,
28          input_data: "10\n"
29              "0 7 2\n"
30              "10 -1 -1\n"
31              "20 -1 6\n"
32              "30 8 9\n"
33              "40 3 -1\n"
34              "50 -1 -1\n"
35              "60 1 -1\n"
36              "70 5 4\n"
37              "80 -1 -1\n"
38              "90 -1 -1\n",
39          expected: "50 70 80 30 90 40 0 20 10 60\n"
40              "0 70 50 40 30 80 90 20 60 10\n"
41              "50 80 90 30 40 70 10 60 20 0",
42      )

45  def test_maximum():  MarieeMorte
46      n = 100_000
47      lines = [str(n)]
48      for i in range(n - 1):
49          lines.append(f"{i} -1 {i + 1}")
50      lines.append(f"{n - 1} -1 -1")
51      input_data = "\n".join(lines) + "\n"
52
53      expected_in = " ".join(str(i) for i in range(n))
54      expected_pre = " ".join(str(i) for i in range(n))
55      expected_post = " ".join(str(i) for i in range(n - 1, -1, -1))
56
57      check( task_num: 1, input_data, expected: f"{expected_in}\n{expected_pre}\n{expected_post}")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.091s	memory=188.0 KB
Первый пример из задачи	time=0.221s	memory=220.0 KB
Второй пример из задачи	time=0.119s	memory=212.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.266s	memory=36.4 MB

Вывод по задаче

Реализованы три классических обхода двоичного дерева «в глубину» без рекурсии, что гарантирует работу на вырожденных деревьях глубиной до 10^5 вершин. Итеративный подход позволил избежать переполнения стека вызовов.

12 Задача. Проверка сбалансированности [2 s, 256 Mb]

Описание задания к задаче

АВЛ-дерево является сбалансированным в следующем смысле: для любой вершины высота её левого поддерева отличается от высоты её правого поддерева не больше, чем на единицу.

Введём понятие баланса вершины: для вершины дерева V её баланс $B(V)$ равен разности высоты правого поддерева и высоты левого поддерева. Таким образом, свойство АВЛ-дерева, приведённое выше, можно сформулировать следующим образом: для любой её вершины V выполняется следующее неравенство:

$$-1 \leq B(V) \leq 1$$

Дано двоичное дерево поиска. Для каждой его вершины требуется определить её баланс.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание двоичного дерева.
В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i + 1)$ -ой строке файла ($1 \leq i \leq N$) находится описание i -ой вершины, состоящее из трёх чисел K_i , L_i , R_i , разделённых пробелами – ключа K_i в i -ой вершине, номера левого L_i ребёнка i -ой вершины ($i < L_i \leq N$ или $L_i = 0$, если левого ребёнка нет) и номера правого R_i ребёнка i -ой вершины ($i < R_i \leq N$ или $R_i = 0$, если правого ребёнка нет). Все ключи различны. Гарантируется, что данное дерево является деревом поиска.
- **Ограничения на входные данные.** $0 \leq N \leq 2 \cdot 10^5$, $|K_i| \leq 10^9$.
- **Формат вывода / выходного файла (output.txt).** Для i -ой вершины в i -ой строке выведите одно число – баланс данной вершины.
- **Ограничение по времени.** 2 сек.
- **Ограничение по памяти.** 256 мб.

Исходный код и описание решения

```
def solve():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n = int(lines[0].strip())
```

```

if n == 0:
    return

left = [0] * n
right = [0] * n

for i in range(n):
    parts = lines[i + 1].strip().split()
    left[i] = int(parts[1]) - 1
    right[i] = int(parts[2]) - 1

heights = [0] * n
balances = [0] * n

stack = [(0, False)]

while stack:
    node, processed = stack.pop()

    if node == -1:
        continue

    if processed:
        left_height = heights[left[node]] if left[node] != -1 else 0
        right_height = heights[right[node]] if right[node] != -1 else 0

        heights[node] = 1 + max(left_height, right_height)

        balances[node] = right_height - left_height
    else:
        stack.append((node, True))

        if right[node] != -1:
            stack.append((right[node], False))
        if left[node] != -1:
            stack.append((left[node], False))

with open('output.txt', 'w') as f:
    for i in range(n):
        f.write(str(balances[i]) + '\n')

if __name__ == '__main__':
    solve()

```

Читаем из input.txt количество вершин n и описания вершин с номерами детей, где 0 означает отсутствие ребёнка. Для каждой вершины требуется вычислить её баланс - разность высот правого и левого поддеревьев.

Используем итеративный пост-обход (сначала дети, потом родитель) со стеком пар (вершина, processed). При обработке вершины вычисляем высоты её детей (или 0, если ребёнка нет), сохраняем высоту вершины в массиве heights и её баланс в массиве balances. Пост-обход гарантирует, что к моменту обработки вершины высоты всех её потомков уже известны. Результаты выводятся для каждой вершины в порядке её номера во входных данных.

Сложность - $O(n)$ по времени и $O(n)$ по памяти, при $n \leq 2 \cdot 10^5$ работает быстро.

Описание проведённых тестов

input.txt		output.txt	
1	6	1	3
2	-2 0 2	2	-1
3	8 4 3	3	0
4	9 0 0	4	0
5	3 6 5	5	0
6	6 0 0	6	0
7	0 0 0	7	
8			

```
4 ▶ def test_minimal():  Ⓜ MarieeMorte
5     check( task_num: 12, input_data: "1\n5 0 0\n", expected: "0")
6
7
8 ▶ def test_example():  Ⓜ MarieeMorte
9     check(
10         task_num: 12,
11         input_data: "6\n"
12         "-2 0 2\n"
13         "8 4 3\n"
14         "9 0 0\n"
15         "3 6 5\n"
16         "6 0 0\n"
17         "0 0 0\n",
18         expected: "3\n-1\n0\n0\n0\n0",
19     )
```

```

22 def test_maximum():
23     n = 200_000
24     lines = [str(n)]
25     for i in range(1, n):
26         lines.append(f"{i} {i + 1} 0")
27     lines.append(f"{n} 0 0")
28     input_data = "\n".join(lines) + "\n"
29
30     expected = []
31     for i in range(1, n + 1):
32         left_h = n - i
33         right_h = 0
34         expected.append(str(right_h - left_h))
35
36     check(task_num: 12, input_data, "\n".join(expected), timeout=60)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.090s	memory=84.0 KB
Пример из задачи	time=0.262s	memory=256.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.551s	memory=41.9 MB

Вывод по задаче

Реализован алгоритм вычисления балансов всех вершин BST за $O(n)$ с использованием итеративного пост-обхода. Алгоритм корректно работает с вырожденными деревьями глубиной до $2 \cdot 10^5$ без переполнения стека.

17 Задача. Множество с суммой [120 s, 512 Mb]

Описание задания к задаче

В этой задаче ваша цель – реализовать структуру данных для хранения набора целых чисел и быстрого вычисления суммы элементов в заданном диапазоне.

Реализуйте такую структуру данных, в которой хранится набор целых чисел S и доступны следующие операции:

- $\text{add}(i)$ – добавить число i в множество S . Если i уже есть в S , то ничего делать не надо;
- $\text{del}(i)$ – удалить число i из множества S . Если i нет в S , то ничего делать не надо;
- $\text{find}(i)$ – проверить, есть ли i во множестве S или нет;
- $\text{sum}(l, r)$ – вывести сумму всех элементов v из S таких, что $l \leq v \leq r$.

- **Формат ввода / входного файла (input.txt).** Изначально множество S пусто. Первая строка содержит n – количество операций. Следующие n строк содержат операции. Однако, чтобы убедиться, что ваше решение может работать в режиме онлайн, каждый запрос фактически будет зависеть от результата последнего запроса суммы. Обозначим $M = 1\,000\,000\,001$. В любой момент пусть x будет результатом последней операции суммирования или просто 0, если до этого операций суммирования не было. Тогда каждая операция будет являться одной из следующих:

- «+ i » – добавить некоторое число в множество S . Но не само число i , а число $((i + x) \bmod M)$.
- «- i » – удалить из множества S , т.е. $\text{del}((i + x) \bmod M)$.
- «? i » – $\text{find}((i + x) \bmod M)$.
- «s l r» – вывести сумму всех элементов множества S из определенного диапазона, т.е. $\text{sum}((l+x) \bmod M, (r+x) \bmod M)$.

- **Ограничения на входные данные.** $1 \leq n \leq 100000$, $1 \leq i \leq 10^9$.
- **Формат вывода / выходного файла (output.txt).** Для каждого запроса «find», выведите только «Found» или «Not found» (без кавычек, первая буква заглавная) в зависимости от того, есть ли число $((i + x) \bmod M)$ в S или нет.

Для каждого запроса суммы «sum» выведите сумму всех значений v из S из диапазона $(l + x) \bmod M \leq v \leq (r + x) \bmod M$, где x – результат подсчета прошлой суммы «sum», или 0, если ещё не было таких операций.

- Ограничение по времени. 120 сек. Python
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
class Node:
    __slots__ = ('key', 'left', 'right', 'parent', 'sum')

    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.parent = None
        self.sum = key

def get_sum(node):
    return node.sum if node else 0

def update_sum(node):
    if node:
        node.sum = node.key + get_sum(node.left) + get_sum(node.right)

def is_left_child(node):
    return node.parent and node.parent.left == node

def is_right_child(node):
    return node.parent and node.parent.right == node

def rotate(x):
    p = x.parent
    g = p.parent

    if is_left_child(x):
        p.left = x.right
        if x.right:
            x.right.parent = p
        x.right = p
    else:
        p.right = x.left
        if x.left:
            x.left.parent = p
        x.left = p

    p.parent = x
    x.parent = g

    if g:
        if g.left == p:
            g.left = x
        else:
            g.right = x

    update_sum(p)
    update_sum(x)
```

```

class SplayTree:
    def __init__(self):
        self.root = None

    def splay(self, x):
        while x.parent:
            p = x.parent
            g = p.parent

            if g:
                if is_left_child(x) == is_left_child(p):
                    rotate(p)
                else:
                    rotate(x)

            rotate(x)

        self.root = x

    def find(self, key):
        if not self.root:
            return None

        current = self.root
        last = None

        while current:
            last = current
            if key == current.key:
                self.splay(current)
                return current
            elif key < current.key:
                current = current.left
            else:
                current = current.right

        if last:
            self.splay(last)
        return None

    def insert(self, key):
        if not self.root:
            self.root = Node(key)
            return

        if self.find(key):
            return

        current = self.root
        parent = None

        while current:
            parent = current
            if key < current.key:
                current = current.left
            else:
                current = current.right

        new_node = Node(key)

```

```

        new_node.parent = parent

        if key < parent.key:
            parent.left = new_node
        else:
            parent.right = new_node

        self.splay(new_node)

def delete(self, key):
    if not self.find(key):
        return

    node = self.root

    left = node.left
    right = node.right

    if left:
        left.parent = None
    if right:
        right.parent = None

    if not left:
        self.root = right
    else:
        max_left = left
        while max_left.right:
            max_left = max_left.right

        self.splay(max_left)
        self.root = max_left
        self.root.right = right
        if right:
            right.parent = self.root
        update_sum(self.root)

def sum_range(self, l, r):
    if l > r:
        return 0

    if not self.root:
        return 0

    if not self.find(l):
        pass

    left, right = self.split(self.root, l)

    mid, right2 = self.split(right, r + 1) if right else (None, None)

    result = get_sum(mid)

    self.root = self.merge(self.merge(left, mid), right2)

    return result

def split(self, node, key):
    if not node:
        return None, None

```

```

        current = node
        last = None

        while current:
            last = current
            if current.key >= key:
                current = current.left
            else:
                current = current.right

        if not last:
            return None, node

        self.splay(last)

        if last.key >= key:
            left = last.left
            if left:
                left.parent = None
            last.left = None
            update_sum(last)
            return left, last
        else:
            right = last.right
            if right:
                right.parent = None
            last.right = None
            update_sum(last)
            return last, right

    def merge(self, left, right):
        if not left:
            return right
        if not right:
            return left

        max_left = left
        while max_left.right:
            max_left = max_left.right

        self.splay(max_left)
        max_left.right = right
        right.parent = max_left
        update_sum(max_left)

        return max_left

def solve():
    tree = SplayTree()
    output = []
    mod = 1000000001
    x = 0

    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n = int(lines[0].strip())

```

```

for i in range(1, n + 1):
    parts = lines[i].strip().split()
    if not parts:
        continue

    op = parts[0]

    if op == '+':
        key = (int(parts[1]) + x) % mod
        tree.insert(key)
    elif op == '-':
        key = (int(parts[1]) + x) % mod
        tree.delete(key)
    elif op == '?':
        key = (int(parts[1]) + x) % mod
        found = tree.find(key) is not None
        output.append('Found' if found else 'Not found')
    elif op == 's':
        l = (int(parts[1]) + x) % mod
        r = (int(parts[2]) + x) % mod
        if l > r:
            l, r = r, l
        result = tree.sum_range(l, r)
        output.append(str(result))
        x = result

with open('output.txt', 'w') as f:
    f.write('\n'.join(output))

if __name__ == '__main__':
    solve()

```

Читаем из input.txt количество операций n и сами операции. Реализуется множество с операциями add, del, find и sum(l , r). Все запросы обрабатываются онлайн: ключ вычисляется как $(i + x) \bmod M$, где x - результат последней суммы, $M = 10^9 + 1$.

Используется Splay-дерево - самобалансирующееся BST, в каждом узле которого хранится сумма ключей поддерева. Операции split и merge позволяют вырезать диапазон $[l, r]$ и получить сумму его элементов за $O(\log n)$ амортизированно. При операции sum(l , r) дерево разделяется на три части: элементы меньше l , элементы от l до r включительно и элементы больше r . Сумма средней части - ответ, после чего дерево собирается обратно. Операции add, del и find выполняются через splay-подъём нужной вершины в корень. Результаты всех запросов find и sum записываются в output.txt.

Сложность - $O(\log n)$ амортизированно на операцию.

Описание проведённых тестов

input.txt		output.txt	
1	15	1	Not found
2	? 1	2	Found
3	+ 1	3	3
4	? 1	4	Found
5	+ 2	5	Not found
6	s 1 2	6	1
7	+ 1000000000	7	Not found
8	? 1000000000	8	10
9	- 1000000000		
10	? 1000000000		
11	s 999999999 1000000000		
12	- 2		
13	? 2		
14	- 0		
15	+ 9		
16	s 0 9		
17			

input.txt		output.txt	
1	5	1	Not found
2	? 0	2	Found
3	+ 0	3	Not found
4	? 0		
5	- 0		
6	? 0		
7			

input.txt			
1	5		
2	+ 491572259		
3	? 491572259		
4	? 899375874		
5	s 310971296 877523306	1	Found
6	+ 352411209	2	Not found
7		3	491572259

```

4  def test_minimal():  MarieeMorte
5      check( task_num: 17, input_data: "1\n? 0\n", expected: "Not found")

```

```
8  ▶ def test_example_1():  Ⓐ MarieeMorte
9      check(
10         task_num: 17,
11         input_data: "15\n"
12         "? 1\n"
13         "+ 1\n"
14         "? 1\n"
15         "+ 2\n"
16         "s 1 2\n"
17         "+ 1000000000\n"
18         "? 1000000000\n"
19         "- 1000000000\n"
20         "? 1000000000\n"
21         "s 999999999 1000000000\n"
22         "- 2\n"
23         "? 2\n"
24         "- 0\n"
25         "+ 9\n"
26         "s 0 9\n",
27         expected: "Not found\n"
28         "Found\n"
29         "3\n"
30         "Found\n"
31         "Not found\n"
32         "1\n"
33         "Not found\n"
34         "10",
35     )
```

```

38 ▶ def test_example_2():  Ⓜ MarieeMorte
39     check(
40         task_num: 17,
41         input_data: "5\n"
42         "? 0\n"
43         "+ 0\n"
44         "? 0\n"
45         "- 0\n"
46         "? 0\n",
47         expected: "Not found\nFound\nNot found",
48     )
49
50
51 ▶ def test_example_3():  Ⓜ MarieeMorte
52     check(
53         task_num: 17,
54         input_data: "5\n"
55         "+ 491572259\n"
56         "? 491572259\n"
57         "? 899375874\n"
58         "s 310971296 877523306\n"
59         "+ 352411209\n",
60         expected: "Found\nNot found\n491572259",
61     )

```

```

38 ▶ def test_maximum():  Ⓜ MarieeMorte
39     n = 100_000
40     lines = [str(n)]
41     for i in range(1, n):
42         lines.append(f"+ {i}")
43     lines.append(f"s 1 {n - 1}")
44     input_data = "\n".join(lines) + "\n"
45     expected = str((n - 1) * n // 2)
46     check(task_num: 17, input_data, expected, timeout=60)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.084s	memory=392.0 KB
Первый пример из задачи	time=0.223s	memory=452.0 KB
Второй пример из задачи	time=0.267s	memory=496.0 KB
Третий пример из задачи	time=0.223s	memory=480.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.637s	memory=21.1 MB

Вывод по задаче

Реализована структура данных на основе Splay-дерева с поддержкой сумм в поддеревьях. Амортизированная логарифмическая сложность операций позволяет обрабатывать до 10^5 запросов в онлайн-режиме, а split и merge дают эффективный доступ к диапазонам.

Дополнительные задачи

3 Задача. Простейшее BST [2 с, 256 Мб, 1 балл, с учётом штрафа 0,5 балла]

Описание задания к задаче

В этой задаче вам нужно написать простейшее BST по явному ключу и отвечать им на запросы:

- «+ x» – добавить в дерево x (если x уже есть, ничего не делать).
- «> x» – вернуть минимальный элемент больше x или 0, если таких нет.
- **Формат ввода / входного файла (input.txt).** В каждой строке содержится один запрос. Все x - целые числа, количество запросов N не указано в начале, не более 300 000. Гарантируется, что все x выбраны равномерным распределением.
- Случайные данные! Не нужно ничего специально балансировать.
- **Ограничения на входные данные.** $1 \leq x \leq 10^9$, $1 \leq N \leq 300000$
- **Формат вывода / выходного файла (output.txt).** Для каждого запроса вида «> x» выведите в отдельной строке ответ.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.

Исходный код и описание решения

```
class Node:
    __slots__ = ('key', 'left', 'right')

    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)

    current = root
    while True:
        if key == current.key:
            return root
        elif key < current.key:
            if current.left is None:
                current.left = Node(key)
                return root
            current = current.left
```

```

        else:
            if current.right is None:
                current.right = Node(key)
                return root
            current = current.right

def find_next(root, key):
    result = 0
    current = root

    while current is not None:
        if current.key > key:
            result = current.key
            current = current.left
        else:
            current = current.right

    return result

def main():
    root = None
    output = []

    with open('input.txt', 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            parts = line.split()
            if len(parts) < 2:
                continue

            op = parts[0]
            x = int(parts[1])

            if op == '+':
                root = insert(root, x)
            else:
                output.append(str(find_next(root, x)))

    with open('output.txt', 'w') as f:
        f.write('\n'.join(output))

if __name__ == '__main__':
    main()

```

Читаем из input.txt строки с командами: «+ x» означает добавление ключа x в дерево, «> x» - поиск минимального элемента, строго большего x. Храним дерево в виде узлов с ключом и ссылками на левого и правого ребёнка.

Вставка выполняется итеративно: если дерево пустое, создаём корень; иначе спускаемся от корня, сравнивая ключ с текущим, и находим место для нового узла. Если ключ уже есть, ничего не делаем. Поиск верхней границы идёт по дереву с сохранением текущего

лучшего кандидата: если ключ текущей вершины больше x , запоминаем его и идём влево, иначе - вправо. Если кандидат не найден, выводим 0.

По условию данные случайны, поэтому балансировка не требуется, и дерево остаётся достаточно сбалансированным. Результаты всех запросов « $> x$ » записываются в output.txt.

Сложность - в среднем $O(\log n)$ на операцию, при $n \leq 300\,000$ работает быстро за счёт случайности данных.

Описание проведённых тестов

input.txt			
1	+ 1		
2	+ 3		
3	+ 3		
4	> 1		
5	> 2		
6	> 3	1	3
7	+ 2	2	3
8	> 1	3	0
9		4	2

```
7 ▶ def test_minimal():  Ⓜ MarieeMorte
8     check( task_num: 3, input_data: "+ 1\n", expected: "")
9
10
11 ▶ def test_example():  Ⓜ MarieeMorte
12     check(
13         task_num: 3,
14         input_data: "+ 1\n"
15         "+ 3\n"
16         "+ 3\n"
17         "> 1\n"
18         "> 2\n"
19         "> 3\n"
20         "+ 2\n"
21         "> 1\n",
22         expected: "3\n3\n0\n2",
23     )
```

```

26 def test_maximum():  Ⓜ MarieeMorte
27     random.seed(42)
28     n = 300_000
29     lines = []
30     added = set()
31
32     for _ in range(n // 2):
33         x = random.randint(a: 1, 10 ** 9)
34         if x not in added:
35             added.add(x)
36             lines.append(f"+ {x}")
37
38     for _ in range(n - len(lines)):
39         x = random.randint(a: 0, 10 ** 9)
40         lines.append(f"> {x}")
41
42     input_data = "\n".join(lines) + "\n"
43
44     sorted_vals = sorted(added)
45     expected = []
46     for line in lines:
47         if line.startswith(">"):
48             x = int(line[2:])
49             idx = bisect.bisect_right(sorted_vals, x)
50             if idx < len(sorted_vals):
51                 expected.append(str(sorted_vals[idx]))
52             else:
53                 expected.append("0")
54
55     check(task_num: 3, input_data, "\n".join(expected), timeout=60)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.085s	memory=20.0 KB
Пример из задачи	time=0.238s	memory=152.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.787s	memory=26.5 MB

Вывод по задаче

Реализовано простое BST с операциями вставки и поиска следующего элемента. Благодаря равномерному распределению входных данных дерево остаётся сбалансированным, что даёт логарифмическую сложность операций.

5 Задача. Простое двоичное дерево поиска [2 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]

Описание задания к задаче

Реализуйте простое двоичное дерево поиска.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание операций с деревом, их количество N не превышает 100. В каждой строке находится одна из следующих операций:
 - `insert x` – добавить в дерево ключ x . Если ключ x есть в дереве, то ничего делать не надо;
 - `delete x` – удалить из дерева ключ x . Если ключа x в дереве нет, то ничего делать не надо;
 - `exists x` – если ключ x есть в дереве выведите «true», если нет – «false»;
 - `next x` – выведите минимальный элемент в дереве, строго больший x , или «none», если такого нет;
 - `prev x` – выведите максимальный элемент в дереве, строго меньший x , или «none», если такого нет.

В дерево помещаются и извлекаются только целые числа, не превышающие по модулю 10^9 .

- **Ограничения на входные данные.** $0 \leq N \leq 100$, $|x_i| \leq 10^9$.
- **Формат вывода / выходного файла (output.txt).** Выведите последовательно результат выполнения всех операций `exists`, `next`, `prev`. Следуйте формату выходного файла из примера.
- **Ограничение по времени.** 2 сек.
- **Ограничение по памяти.** 512 мб.

Исходный код и описание решения

```
class Node:
    __slots__ = ('key', 'left', 'right')

    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
```

```

        self.root = None

def insert(self, key):
    if self.root is None:
        self.root = Node(key)
        return

    current = self.root
    while True:
        if key == current.key:
            return
        elif key < current.key:
            if current.left is None:
                current.left = Node(key)
                return
            current = current.left
        else:
            if current.right is None:
                current.right = Node(key)
                return
            current = current.right

def delete(self, key):
    parent = None
    current = self.root

    while current is not None and current.key != key:
        parent = current
        if key < current.key:
            current = current.left
        else:
            current = current.right

    if current is None:
        return

    if current.left is None and current.right is None:
        if parent is None:
            self.root = None
        elif parent.left == current:
            parent.left = None
        else:
            parent.right = None

    elif current.left is None:
        if parent is None:
            self.root = current.right
        elif parent.left == current:
            parent.left = current.right
        else:
            parent.right = current.right

    elif current.right is None:
        if parent is None:
            self.root = current.left
        elif parent.left == current:
            parent.left = current.left
        else:
            parent.right = current.left

    else:

```

```

        successor_parent = current
        successor = current.right
        while successor.left is not None:
            successor_parent = successor
            successor = successor.left

        current.key = successor.key

        if successor_parent.left == successor:
            successor_parent.left = successor.right
        else:
            successor_parent.right = successor.right

    def exists(self, key):
        current = self.root
        while current is not None:
            if key == current.key:
                return True
            elif key < current.key:
                current = current.left
            else:
                current = current.right
        return False

    def next(self, key):
        result = None
        current = self.root

        while current is not None:
            if current.key > key:
                result = current.key
                current = current.left
            else:
                current = current.right

        return result

    def prev(self, key):
        result = None
        current = self.root

        while current is not None:
            if current.key < key:
                result = current.key
                current = current.right
            else:
                current = current.left

        return result

def main():
    tree = BST()
    output = []

    with open('input.txt', 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

```

```

        parts = line.split()
        if len(parts) < 2:
            continue

        op = parts[0]
        x = int(parts[1])

        if op == 'insert':
            tree.insert(x)
        elif op == 'delete':
            tree.delete(x)
        elif op == 'exists':
            output.append('true' if tree.exists(x) else 'false')
        elif op == 'next':
            res = tree.next(x)
            output.append(str(res) if res is not None else 'none')
        elif op == 'prev':
            res = tree.prev(x)
            output.append(str(res) if res is not None else 'none')

    with open('output.txt', 'w') as f:
        f.write('\n'.join(output))

if __name__ == '__main__':
    main()

```

Читаем из input.txt строки с командами: insert x, delete x, exists x, next x, prev x. Все операции выполнены итеративно, дерево хранится в виде узлов с ключом и ссылками на детей.

Вставка выполняется спуском от корня с поиском места для нового узла; если ключ уже есть, ничего не делаем. Удаление обрабатывает три случая: если узел - лист, просто обнуляем ссылку у родителя; если у узла один ребёнок, подвешиваем его к родителю удаляемого узла; если два ребёнка, находим минимальный узел в правом поддереве (преемника), копируем его ключ в удаляемый узел и удаляем преемника, у которого гарантированно нет левого ребёнка. Поиск exists возвращает true или false. Поиск next и prev выполняется спуском по дереву с сохранением лучшего кандидата: для next ищем минимальный элемент, строго больший x, для prev - максимальный, строго меньший x. Результаты всех запросов exists, next и prev записываются в output.txt.

Сложность - $O(h)$ на операцию, где h - высота дерева.

Описание проведённых тестов

input.txt		output.txt	
1	insert 2	1	true
2	insert 5	2	false
3	insert 3	3	5
4	exists 2	4	3
5	exists 4	5	none
6	next 4	6	3
7	prev 4		
8	delete 5		
9	next 4		
10	prev 4		
11			

```
4 ▶ def test_minimal():  Ⓜ MarieeMorte
5     check( task_num: 5,  input_data: "exists 0\n",  expected: "false")
6
7
8 ▶ def test_example():  Ⓜ MarieeMorte
9     check(
10         task_num: 5,
11         input_data: "insert 2\n"
12         "insert 5\n"
13         "insert 3\n"
14         "exists 2\n"
15         "exists 4\n"
16         "next 4\n"
17         "prev 4\n"
18         "delete 5\n"
19         "next 4\n"
20         "prev 4\n",
21         expected: "true\nfalse\n5\n3\nnone\n3",
22     )
```

```

25 ▶ def test_maximum():  👤 MarieeMorte
26     lines = []
27     for i in range(50):
28         lines.append(f"insert {i}")
29     for i in range(50):
30         lines.append(f"exists {i}")
31     for i in range(49):
32         lines.append(f"next {i}")
33     for i in range(1, 50):
34         lines.append(f"prev {i}")
35     input_data = "\n".join(lines) + "\n"
36
37     expected = []
38     for i in range(50):
39         expected.append("true")
40     for i in range(49):
41         expected.append(str(i + 1))
42     for i in range(1, 50):
43         expected.append(str(i - 1))
44
45     check(task_num: 5, input_data, "\n".join(expected))

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.221s	memory=316.0 KB
Пример из задачи	time=0.232s	memory=384.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.261s	memory=476.0 KB

Вывод по задаче

Реализовано полноценное BST со всеми базовыми операциями. Итеративный подход и аккуратная обработка трёх случаев удаления обеспечивают корректную работу на всех входных данных, включая пустое дерево и удаление корня.

7 Задача. Опознавание двоичного дерева поиска (усложненная версия)

[10 s, 512 Mb, 2.5 балла, с учётом штрафа 2 балла]

Описание задания к задаче

Вам дано двоичное дерево с ключами - целыми числами, которые могут повторяться. Вам нужно проверить, является ли это правильным двоичным деревом поиска. Теперь, для каждой вершины дерева V выполняется следующее условие:

- все ключи вершин из левого поддеревья меньше ключа вершины V ;
- все ключи вершин из правого поддеревья **больше или равны** ключу вершины V .

Другими словами, узлы с меньшими ключами находятся слева, а узлы с большими ключами – справа, дубликаты всегда справа. Вам необходимо проверить, удовлетворяет ли данная структура двоичного дерева этому условию.

- **Формат ввода / входного файла (input.txt).** В первой строке входного файла содержится количество узлов n . Узлы дерева пронумерованы от 0 до $n - 1$. Узел 0 является корнем.

Следующие n строк содержат информацию об узлах 0, 1, ..., $n - 1$ по порядку. Каждая из этих строк содержит три целых числа K_i , L_i и R_i . K_i – ключ i -го узла, L_i – индекс левого ребёнка i -го узла, а R_i – индекс правого ребёнка i -го узла. Если у i -го узла нет левого или правого ребёнка (или обоих), соответствующие числа L_i или R_i (или оба) будут равны -1 .

- **Ограничения на входные данные.** $0 \leq n \leq 10^5$, $-2^{31} \leq K_i \leq 2^{31} - 1$, $-1 \leq L_i, R_i \leq n - 1$. Гарантируется, что данное дерево является двоичным деревом. В частности, если $L_i \neq -1$ и $R_i \neq -1$, то $L_i \neq R_i$. Кроме того, узел не может быть ребёнком двух разных узлов. Кроме того, каждый узел является потомком корневого узла. Обратите внимание, что минимальное и максимальное возможные значения 32-битного целочисленного типа могут быть ключами в дереве.
- **Формат вывода / выходного файла (output.txt).** Если заданное двоичное дерево является правильным двоичным деревом поиска, выведите одно слово «CORRECT» (без кавычек). В противном случае выведите одно слово «INCORRECT» (без кавычек).
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
def solve():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n = int(lines[0].strip())

    if n == 0:
        with open('output.txt', 'w') as f:
            f.write('CORRECT\n')
        return

    keys = [0] * n
    left = [0] * n
    right = [0] * n

    for i in range(n):
        parts = lines[i + 1].strip().split()
        keys[i] = int(parts[0])
        left[i] = int(parts[1])
        right[i] = int(parts[2])

    stack = [(0, -2 ** 31, 2 ** 31 - 1)]
    result = True

    while stack:
        node, min_key, max_key = stack.pop()

        if node == -1:
            continue

        key = keys[node]

        if key < min_key or key > max_key:
            result = False
            break

        stack.append((left[node], min_key, key - 1))
        stack.append((right[node], key, max_key))

    with open('output.txt', 'w') as f:
        f.write('CORRECT\n' if result else 'INCORRECT\n')

if __name__ == '__main__':
    solve()
```

Читаем из `input.txt` количество узлов n и описания вершин: ключ K_i , индекс левого L_i и правого R_i ребёнка. Требуется проверить, что дерево является BST с дубликатами: в левом поддереве все ключи строго меньше, в правом - больше или равны ключу вершины.

Используем итеративную проверку с явным стеком: в стеке хранятся тройки (вершина, min_key, max_key). Для каждой вершины проверяем, что её ключ лежит в допустимом диапазоне. При спуске в левое поддерево верхняя граница становится key - 1 (строго меньше), при спуске в правое - нижняя граница становится key (больше или равно). Начальные границы - -2^{31} и $2^{31} - 1$, чтобы корректно работать с минимальным и максимальным значениями 32-битного целочисленного типа. Если хотя бы одна вершина нарушает условие, выводим INCORRECT, иначе CORRECT.

Сложность - $O(n)$, при $n \leq 10^5$ работает мгновенно.

Описание проведённых тестов

input.txt		output.txt	
1	3	1	CORRECT
2	2 1 2	2	
3	1 -1 -1		
4	3 -1 -1		
5			

input.txt		output.txt	
1	3	1	INCORRECT
2	1 1 2	2	
3	2 -1 -1		
4	3 -1 -1		
5			

input.txt		output.txt	
1	3	1	CORRECT
2	2 1 2	2	
3	1 -1 -1		
4	2 -1 -1		
5			

input.txt ✕			
1	3		
2	2 1 2		
3	2 -1 -1		
4	3 -1 -1		
5			

output.txt ✕	
1	INCORRECT
2	

input.txt ✕			
1	5		
2	1 -1 1		
3	2 -1 2		
4	3 -1 3		
5	4 -1 4		
6	5 -1 -1		
7			

output.txt ✕	
1	CORRECT
2	

input.txt ✕			
1	7		
2	4 1 2		
3	2 3 4		
4	6 5 6		
5	1 -1 -1		
6	3 -1 -1		
7	5 -1 -1		
8	7 -1 -1		
9			

output.txt ✕	
1	CORRECT
2	

input.txt		output.txt	
1	1	1	CORRECT
2	2147483647 -1 -1	2	
3			

```

4 ▶ def test_minimal():  Ⓜ MarieeMorte
5     check( task_num: 7, input_data: "0\n", expected: "CORRECT")
6
7
8 ▶ def test_example_1():  Ⓜ MarieeMorte
9     check( task_num: 7, input_data: "3\n2 1 2\n1 -1 -1\n3 -1 -1\n", expected: "CORRECT")
10
11
12 ▶ def test_example_2():  Ⓜ MarieeMorte
13     check( task_num: 7, input_data: "3\n1 1 2\n2 -1 -1\n3 -1 -1\n", expected: "INCORRECT")
14
15
16 ▶ def test_example_3():  Ⓜ MarieeMorte
17     check( task_num: 7, input_data: "3\n2 1 2\n1 -1 -1\n2 -1 -1\n", expected: "CORRECT")
18
19
20 ▶ def test_example_4():  Ⓜ MarieeMorte
21     check( task_num: 7, input_data: "3\n2 1 2\n2 -1 -1\n3 -1 -1\n", expected: "INCORRECT")
22
23
24 ▶ def test_example_5():  Ⓜ MarieeMorte
25     check( task_num: 7, input_data: "5\n1 -1 1\n2 -1 2\n3 -1 3\n4 -1 4\n5 -1 -1\n", expected: "CORRECT")

```

```

28 ▶ def test_example_6():  Ⓜ MarieeMorte
29     check(
30         task_num: 7,
31         input_data: "7\n"
32         "4 1 2\n"
33         "2 3 4\n"
34         "6 5 6\n"
35         "1 -1 -1\n"
36         "3 -1 -1\n"
37         "5 -1 -1\n"
38         "7 -1 -1\n",
39         expected: "CORRECT",
40     )
41
42
43 ▶ def test_example_7():  Ⓜ MarieeMorte
44     check( task_num: 7, input_data: "1\n2147483647 -1 -1\n", expected: "CORRECT")
45
46
47 ▶ def test_maximum():  Ⓜ MarieeMorte
48     n = 100_000
49     lines = [str(n)]
50     for i in range(n - 1):
51         lines.append(f"{i} -1 {i + 1}")
52     lines.append(f"{n - 1} -1 -1")
53     input_data = "\n".join(lines) + "\n"
54     check( task_num: 7, input_data, expected: "CORRECT")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.085s	memory=36.0 KB
Первый пример из задачи	time=0.209s	memory=132.0 KB
Второй пример из задачи	time=0.093s	memory=68.0 KB
Третий пример из задачи	time=0.095s	memory=60.0 KB
Четвёртый пример из задачи	time=0.086s	memory=40.0 KB
Пятый пример из задачи	time=0.085s	memory=128.0 KB
Шестой пример из задачи	time=0.086s	memory=100.0 KB
Седьмой пример из задачи	time=0.095s	memory=92.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.215s	memory=24.2 MB

Вывод по задаче

Реализован алгоритм проверки BST с поддержкой дубликатов, корректно работающий с граничными значениями 32-битного целочисленного типа. Итеративная реализация позволила обработать деревья глубиной до 10^5 без переполнения стека.

8 Задача. Высота дерева возвращается [2 s, 256 Mb, 2 балла, с учётом штрафа 1,5 балла]

Описание задания к задаче

Высотой дерева называется максимальное число вершин дерева в цепочке, начинающейся в корне дерева, заканчивающейся в одном из его листьев, и не содержащей никакой вершину дважды.

Так, высота дерева, состоящего из единственной вершины, равна единице. Высота пустого дерева равна нулю.

Дано двоичное дерево поиска. В вершинах этого дерева записаны ключи – целые числа, по модулю не превышающие 10^9 . Для каждой вершины дерева V выполняется следующее условие:

- все ключи вершин из левого поддеревья меньше ключа вершины V ;
- все ключи вершин из правого поддеревья больше ключа вершины V .

Найдите высоту данного дерева.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание двоичного дерева. В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i + 1)$ -ой строке файла ($1 \leq i \leq N$) находится описание i -ой вершины, состоящее из трёх чисел K_i, L_i, R_i , разделённых пробелами – ключа K_i в i -ой вершине, номера левого L_i ребёнка i -ой вершины ($i < L_i \leq N$ или $L_i = 0$, если левого ребёнка нет) и номера правого R_i ребёнка i -ой вершины ($i < R_i \leq N$ или $R_i = 0$, если правого ребёнка нет).
- **Ограничения на входные данные.** $0 \leq N \leq 2 \cdot 10^5$, $|K_i| \leq 10^9$. Все ключи различны. Гарантируется, что данное дерево является деревом поиска.
- **Формат вывода / выходного файла (output.txt).** Выведите одно целое число – высоту дерева.
- **Ограничение по времени.** 2 сек.
- **Ограничение по памяти.** 256 мб.

Исходный код и описание решения

```
def solve():
    with open('input.txt', 'r') as f:
        lines = f.readlines()
```

```

if not lines:
    return

n = int(lines[0].strip())

if n == 0:
    with open('output.txt', 'w') as f:
        f.write('0\n')
    return

left = [0] * n
right = [0] * n

for i in range(n):
    parts = lines[i + 1].strip().split()
    left[i] = int(parts[1]) - 1
    right[i] = int(parts[2]) - 1

stack = [(0, 1)]
max_height = 0

while stack:
    node, depth = stack.pop()

    if node == -1:
        continue

    max_height = max(max_height, depth)

    if left[node] != -1:
        stack.append((left[node], depth + 1))
    if right[node] != -1:
        stack.append((right[node], depth + 1))

    with open('output.txt', 'w') as f:
        f.write(str(max_height) + '\n')

if __name__ == '__main__':
    solve()

```

Читаем из input.txt количество вершин n и описания вершин с номерами детей, где 0 означает отсутствие ребёнка. Высота дерева - максимальное число вершин на пути от корня до листа. Пустое дерево имеет высоту 0, дерево из одной вершины - 1.

Используем итеративный DFS с явным стеком: в стеке хранятся пары (вершина, глубина). Начальное значение - (корень, 1). На каждой итерации извлекаем вершину и её глубину, обновляем максимум и добавляем детей с глубиной на единицу больше. Если у вершины нет детей, она является листом, и её глубина уже учтена в максимуме. Результат записываем в output.txt.

Сложность - $O(n)$ по времени и $O(n)$ по памяти для стека и массивов дерева.

Описание проведённых тестов

input.txt			
1	6		
2	-2 0 2		
3	8 4 3		
4	9 0 0		
5	3 6 5		
6	6 0 0		
7	0 0 0	1	4
8		2	

```
4 ▶ def test_minimal():  Ⓜ MarieeMorte
5     check(task_num: 8, input_data: "1\n5 0 0\n", expected: "1")
6
7
8 ▶ def test_example():  Ⓜ MarieeMorte
9     check(
10         task_num: 8,
11         input_data: "6\n"
12         "-2 0 2\n"
13         "8 4 3\n"
14         "9 0 0\n"
15         "3 6 5\n"
16         "6 0 0\n"
17         "0 0 0\n",
18         expected: "4",
19     )
20
21
22 ▶ def test_maximum():  Ⓜ MarieeMorte
23     n = 200_000
24     lines = [str(n)]
25     for i in range(1, n):
26         lines.append(f"{i} 0 {i + 1}")
27     lines.append(f"{n} 0 0")
28     input_data = "\n".join(lines) + "\n"
29     check(task_num: 8, input_data, str(n), timeout=60)
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.087s	memory=32.0 KB
Пример из задачи	time=0.225s	memory=80.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.488s	memory=22.4 MB

Вывод по задаче

Реализован алгоритм вычисления высоты дерева за $O(n)$ с использованием итеративного обхода, что позволяет обрабатывать деревья глубиной до 10^5 без риска переполнения стека вызовов.

10 Задача. Проверка корректности [2 s, 256 Mb, 2 балла, с учётом штрафа 1,5 балла]

Описание задания к задаче

Свойство двоичного дерева поиска можно сформулировать следующим образом: для каждой вершины дерева выполняется следующее условие:

- все ключи вершин из левого поддерева меньше ключа вершины V ;
- все ключи вершин из правого поддерева больше ключа вершины V .

Дано двоичное дерево. Проверьте, выполняется ли для него свойство двоичного дерева поиска.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание двоичного дерева.

В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i + 1)$ -ой строке файла ($1 \leq i \leq N$) находится описание i -ой вершины, состоящее из трёх чисел K_i, L_i, R_i , разделённых пробелами – ключа K_i в i -ой вершине, номера левого L_i ребёнка i -ой вершины ($i < L_i \leq N$ или $L_i = 0$, если левого ребёнка нет) и номера правого R_i ребёнка i -ой вершины ($i < R_i \leq N$ или $R_i = 0$, если правого ребёнка нет).

- **Ограничения на входные данные.** $0 \leq N \leq 2 \cdot 10^5$, $|K_i| \leq 10^9$.
- На 60% от при $0 \leq N \leq 2000$.
- **Формат вывода / выходного файла (output.txt).** Выведите «YES», если данное во входном файле дерево является двоичным деревом поиска, и «NO», если не является.
- **Ограничение по времени. 2 сек.**
- **Ограничение по памяти. 256 мб.**

Исходный код и описание решения

```
def solve():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n = int(lines[0].strip())

    if n == 0:
        with open('output.txt', 'w') as f:
```

```

        f.write('YES\n')
    return

    keys = [0] * n
    left = [0] * n
    right = [0] * n

    for i in range(n):
        parts = lines[i + 1].strip().split()
        keys[i] = int(parts[0])
        left[i] = int(parts[1]) - 1
        right[i] = int(parts[2]) - 1

    stack = [(0, -10 ** 18, 10 ** 18)]

    while stack:
        node, min_key, max_key = stack.pop()

        if node == -1:
            continue

        key = keys[node]

        if key <= min_key or key >= max_key:
            with open('output.txt', 'w') as f:
                f.write('NO\n')
            return

        stack.append((left[node], min_key, key))

        stack.append((right[node], key, max_key))

    with open('output.txt', 'w') as f:
        f.write('YES\n')

if __name__ == '__main__':
    solve()

```

Читаем из input.txt количество вершин n и описания вершин, где 0 означает отсутствие ребёнка. Требуется проверить, что дерево является BST без дубликатов: в левом поддереве строго меньше, в правом - строго больше.

Используем итеративный обход с явным стеком, где каждая запись - (вершина, $\min_key$, $\max_key$). Для каждой вершины проверяем, что её ключ строго больше $\min_key$ и строго меньше $\max_key$. Левое поддерево получает диапазон ($\min_key$, key), правое - (key , $\max_key$). Начальные границы - -10^{18} и 10^{18} , чтобы гарантированно перекрыть все возможные значения ключей. Пустое дерево считается корректным BST. Если условие нарушено хотя бы в одной вершине, выводим NO, иначе YES.

Сложность - $O(n)$, при $n \leq 2 \cdot 10^5$ работает за доли секунды.

Описание проведённых тестов

input.txt		output.txt	
1	6	1	YES
2	-2 0 2	2	
3	8 4 3		
4	9 0 0		
5	3 6 5		
6	6 0 0		
7	0 0 0		
8			

input.txt		output.txt	
1	0	1	YES
2		2	

input.txt		output.txt	
1	3	1	NO
2	5 2 3	2	
3	6 0 0		
4	4 0 0		
5			

```

4 ▶ def test_minimal():  🧑 MarieeMorte
5     check( task_num: 10,  input_data: "1\n5 0 0\n",  expected: "YES")
6
7
8 ▶ def test_example_1():  🧑 MarieeMorte
9     check(
10         task_num: 10,
11         input_data: "6\n"
12         "-2 0 2\n"
13         "8 4 3\n"
14         "9 0 0\n"
15         "3 6 5\n"
16         "6 0 0\n"
17         "0 0 0\n",
18         expected: "YES",
19     )
20
21
22 ▶ def test_example_2():  🧑 MarieeMorte
23     check( task_num: 10,  input_data: "0\n",  expected: "YES")
24
25
26 ▶ def test_example_3():  🧑 MarieeMorte
27     check(
28         task_num: 10,
29         input_data: "3\n"
30         "5 2 3\n"
31         "6 0 0\n"
32         "4 0 0\n",
33         expected: "NO",
34     )
35
36
37 ▶ def test_maximum():  🧑 MarieeMorte
38     n = 200_000
39     lines = [str(n)]
40     for i in range(1, n):
41         lines.append(f"{i} 0 {i + 1}")
42     lines.append(f"{n} 0 0")
43     input_data = "\n".join(lines) + "\n"
44     check( task_num: 10, input_data,  expected: "YES", timeout=60)

```

	Время выполнения	Затраты памяти
--	------------------	----------------

Нижняя граница диапазона значений входных данных из текста задачи	time=0.080s	memory=8.0 KB
Первый пример из задачи	time=0.232s	memory=72.0 KB
Второй пример из задачи	time=0.219s	memory=20.0 KB
Третий пример из задачи	time=0.244s	memory=60.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.461s	memory=44.2 MB

Вывод по задаче

Реализован эффективный алгоритм проверки BST, корректно обрабатывающий пустое дерево и деревья глубиной до $2 \cdot 10^5$. Итеративная реализация исключает переполнение стека при работе с вырожденными деревьями.

11 Задача. Сбалансированное двоичное дерево поиска [2 s, 512 Mb, 2 балла, с учётом штрафа 1,5 балла]

Описание задания к задаче

Реализуйте сбалансированное двоичное дерево поиска.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание операций с деревом, их количество N не превышает 10^5 . В каждой строке находится одна из следующих операций:
 - `insert x` – добавить в дерево ключ x . Если ключ x есть в дереве, то ничего делать не надо;
 - `delete x` – удалить из дерева ключ x . Если ключа x в дереве нет, то ничего делать не надо;
 - `exists x` – если ключ x есть в дереве выведите «true», если нет – «false»;
 - `next x` – выведите минимальный элемент в дереве, строго больший x , или «none», если такого нет;
 - `prev x` – выведите максимальный элемент в дереве, строго меньший x , или «none», если такого нет.

В дерево помещаются и извлекаются только целые числа, не превышающие по модулю 10^9 .

- **Ограничения на входные данные.** $0 \leq N \leq 10^5$, $|x_i| \leq 10^9$.
- **Формат вывода / выходного файла (output.txt).** Выведите последовательно результат выполнения всех операций `exists`, `next`, `prev`. Следуйте формату выходного файла из примера.
- **Ограничение по времени.** 2 сек.
- **Ограничение по памяти.** 512 мб.

Исходный код и описание решения

```
class Node:
    __slots__ = ('key', 'left', 'right', 'height')

    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

def get_height(node):
```

```

    return node.height if node else 0

def update_height(node):
    if node:
        node.height = 1 + max(get_height(node.left),
                               get_height(node.right))

def get_balance(node):
    return get_height(node.right) - get_height(node.left) if node else 0

def rotate_left(x):
    y = x.right
    temp = y.left

    y.left = x
    x.right = temp

    update_height(x)
    update_height(y)

    return y

def rotate_right(x):
    y = x.left
    temp = y.right

    y.right = x
    x.left = temp

    update_height(x)
    update_height(y)

    return y

def balance_tree(node):
    if not node:
        return node

    update_height(node)
    balance_factor = get_balance(node)

    if balance_factor > 1:
        if get_balance(node.right) < 0:
            node.right = rotate_right(node.right)
        return rotate_left(node)

    if balance_factor < -1:
        if get_balance(node.left) > 0:
            node.left = rotate_left(node.left)
        return rotate_right(node)

    return node

def find_min(node):
    while node.left:

```

```

        node = node.left
    return node

class AVLTree:
    def __init__(self):
        self.root = None

    def insert(self, node, key):
        if not node:
            return Node(key)

        if key < node.key:
            node.left = self.insert(node.left, key)
        elif key > node.key:
            node.right = self.insert(node.right, key)
        else:
            return node

        return balance_tree(node)

    def delete(self, node, key):
        if not node:
            return node

        if key < node.key:
            node.left = self.delete(node.left, key)
        elif key > node.key:
            node.right = self.delete(node.right, key)
        else:
            if not node.left:
                return node.right
            if not node.right:
                return node.left

            successor = find_min(node.right)
            node.key = successor.key
            node.right = self.delete(node.right, successor.key)

        return balance_tree(node)

    def exists(self, key):
        current = self.root
        while current:
            if key == current.key:
                return True
            elif key < current.key:
                current = current.left
            else:
                current = current.right
        return False

    def next(self, key):
        result = None
        current = self.root

        while current:
            if current.key > key:
                result = current.key
                current = current.left
            else:

```

```

        current = current.right

    return result

def prev(self, key):
    result = None
    current = self.root

    while current:
        if current.key < key:
            result = current.key
            current = current.right
        else:
            current = current.left

    return result

def solve():
    tree = AVLTree()
    output = []

    with open('input.txt', 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            parts = line.split()
            if len(parts) < 2:
                continue

            op = parts[0]
            x = int(parts[1])

            if op == 'insert':
                tree.root = tree.insert(tree.root, x)
            elif op == 'delete':
                tree.root = tree.delete(tree.root, x)
            elif op == 'exists':
                output.append('true' if tree.exists(x) else 'false')
            elif op == 'next':
                res = tree.next(x)
                output.append(str(res) if res is not None else 'none')
            elif op == 'prev':
                res = tree.prev(x)
                output.append(str(res) if res is not None else 'none')

    with open('output.txt', 'w') as f:
        f.write('\n'.join(output))

if __name__ == '__main__':
    solve()

```

Читаем из input.txt строки с командами: insert x, delete x, exists x, next x, prev x. Требуется реализовать AVL-дерево - самобалансирующееся BST, в котором для каждой вершины высоты левого и правого поддеревьев отличаются не более чем на единицу.

Каждый узел хранит ключ, ссылки на детей и высоту поддерева. После каждой вставки и удаления выполняется балансировка: вычисляется баланс вершины (разность высот правого и левого поддеревьев), и при выходе за диапазон $[-1, 1]$ выполняются малый или большой поворот. Малый левый поворот применяется, когда правое поддерево перевешено и его баланс неотрицателен; большой левый поворот - когда баланс правого ребёнка отрицателен, что требует предварительного правого поворота. Аналогично для правых поворотов. Удаление обрабатывает три случая: лист, один ребёнок, два ребёнка (с поиском минимального узла в правом поддереве). Результаты всех запросов `exists`, `next` и `prev` записываются в `output.txt`.

Сложность - $O(\log n)$ на каждую операцию благодаря балансировке.

Описание проведённых тестов

input.txt			
1	insert 2		
2	insert 5		
3	insert 3		
4	exists 2		
5	exists 4		
6	next 4	1	true
7	prev 4	2	false
8	delete 5	3	5
9	next 4	4	3
10	prev 4	5	none
11		6	3

```

4  ▶ def test_minimal():  ⤵ MarieeMorte
5      check( task_num: 11, input_data: "exists 0\n", expected: "false")
6
7
8  ▶ def test_example():  ⤵ MarieeMorte
9      check(
10         task_num: 11,
11         input_data: "insert 2\n"
12         "insert 5\n"
13         "insert 3\n"
14         "exists 2\n"
15         "exists 4\n"
16         "next 4\n"
17         "prev 4\n"
18         "delete 5\n"
19         "next 4\n"
20         "prev 4\n",
21         expected: "true\nfalse\n5\n3\nnone\n3",
22     )

```

```

25 ▶ def test_maximum():  ⤵ MarieeMorte
26     n = 100_000
27     lines = []
28     for i in range(n):
29         lines.append(f"insert {i}")
30     for i in range(n):
31         lines.append(f"exists {i}")
32     for i in range(0, n - 1):
33         lines.append(f"next {i}")
34     for i in range(1, n):
35         lines.append(f"prev {i}")
36     input_data = "\n".join(lines) + "\n"
37
38     expected = []
39     for i in range(n):
40         expected.append("true")
41     for i in range(0, n - 1):
42         expected.append(str(i + 1))
43     for i in range(1, n):
44         expected.append(str(i - 1))
45
46     check( task_num: 11, input_data, "\n".join(expected), timeout=60)

```

	Время выполнения	Затраты памяти
--	------------------	----------------

Нижняя граница диапазона значений входных данных из текста задачи	time=0.084s	memory=444.0 KB
Пример из задачи	time=0.257s	memory=472.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=1.596s	memory=27.3 MB

Вывод по задаче

Реализовано AVL-дерево с логарифмической сложностью всех операций. Балансировка через повороты гарантирует высоту $O(\log n)$, что делает структуру эффективной для 10^5 операций и устойчивой к худшим случаям.

16 Задача. К-й максимум [2 s, 512 Mb, 3 балла, с учётом штрафа 2,5 балла]

Описание задания к задаче

Напишите программу, реализующую структуру данных, позволяющую добавлять и удалять элементы, а также находить k -й максимум.

- **Формат ввода / входного файла (input.txt).** Первая строка входного файла содержит натуральное число n – количество команд. Последующие n строк содержат по одной команде каждая. Команда записывается в виде двух чисел c_i и k_i – тип и аргумент команды соответственно. Поддерживаемые команды:
 - $+1$ (или просто 1): Добавить элемент с ключом k_i .
 - 0 : Найти и вывести k_i -й максимум.
 - -1 : Удалить элемент с ключом k_i .

Гарантируется, что в процессе работы в структуре не требуется хранить элементы с равными ключами или удалять несуществующие элементы. Также гарантируется, что при запросе k_i -го максимума, он существует.

- **Ограничения на входные данные.** $n \leq 100000$, $|k_i| \leq 10^9$.
- **Формат вывода / выходного файла (output.txt).** Для каждой команды нулевого типа в выходной файл должна быть выведена строка, содержащая единственное число – k_i -й максимум.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
class Node:
    __slots__ = ('key', 'left', 'right', 'height', 'size')

    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1
        self.size = 1

def get_height(node):
    return node.height if node else 0

def get_size(node):
```

```

    return node.size if node else 0

def update_height(node):
    if node:
        node.height = 1 + max(get_height(node.left),
                               get_height(node.right))

def update_size(node):
    if node:
        node.size = 1 + get_size(node.left) + get_size(node.right)

def update_node(node):
    update_height(node)
    update_size(node)

def get_balance(node):
    return get_height(node.right) - get_height(node.left) if node else 0

def rotate_left(x):
    y = x.right
    temp = y.left

    y.left = x
    x.right = temp

    update_node(x)
    update_node(y)

    return y

def rotate_right(x):
    y = x.left
    temp = y.right

    y.right = x
    x.left = temp

    update_node(x)
    update_node(y)

    return y

def balance_tree(node):
    if not node:
        return node

    update_node(node)
    balance = get_balance(node)

    if balance > 1:
        if get_balance(node.right) < 0:
            node.right = rotate_right(node.right)
        return rotate_left(node)

```

```

    if balance < -1:
        if get_balance(node.left) > 0:
            node.left = rotate_left(node.left)
        return rotate_right(node)

    return node

def find_min(node):
    while node.left:
        node = node.left
    return node

class AVLTree:
    def __init__(self):
        self.root = None

    def insert(self, node, key):
        if not node:
            return Node(key)

        if key < node.key:
            node.left = self.insert(node.left, key)
        elif key > node.key:
            node.right = self.insert(node.right, key)
        else:
            return node

        return balance_tree(node)

    def delete(self, node, key):
        if not node:
            return node

        if key < node.key:
            node.left = self.delete(node.left, key)
        elif key > node.key:
            node.right = self.delete(node.right, key)
        else:
            if not node.left:
                return node.right
            if not node.right:
                return node.left

            successor = find_min(node.right)
            node.key = successor.key
            node.right = self.delete(node.right, successor.key)

        return balance_tree(node)

    def kth_maximum(self, k):
        current = self.root
        kth_min = get_size(self.root) - k + 1

        while current:
            left_size = get_size(current.left)

            if kth_min == left_size + 1:
                return current.key
            elif kth_min <= left_size:

```

```

        current = current.left
    else:
        kth_min -= left_size + 1
        current = current.right

    return None

def solve():
    tree = AVLTree()
    output = []

    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n = int(lines[0].strip())

    for i in range(1, n + 1):
        parts = lines[i].strip().split()
        if not parts:
            continue

        cmd = int(parts[0])
        key = int(parts[1])

        if cmd == 1:
            tree.root = tree.insert(tree.root, key)
        elif cmd == -1:
            tree.root = tree.delete(tree.root, key)
        elif cmd == 0:
            result = tree.kth_maximum(key)
            output.append(str(result))

    with open('output.txt', 'w') as f:
        f.write('\n'.join(output))

if __name__ == '__main__':
    solve()

```

Читаем из input.txt количество команд n , затем n строк с командами: $+1\ k$ добавляет элемент с ключом k , $-1\ k$ удаляет элемент, $0\ k$ ищет k -й максимум. Гарантируется, что в дереве нет дубликатов, а при запросе k -го максимума он существует.

Используется AVL-дерево, в каждом узле которого дополнительно хранится размер поддерева. Это позволяет искать k -й максимум за $O(\log n)$: преобразуем запрос в поиск $(total_size - k + 1)$ -го минимума и спускаемся по дереву, сравнивая k с размером левого поддерева. Если k равен размеру левого поддерева плюс один, текущая вершина - искомая; если k меньше или равен размеру левого поддерева, идём влево; иначе уменьшаем k на размер левого поддерева плюс один и идём вправо. При вставке и

удалении размеры обновляются вместе с высотами, балансировка выполняется через повороты. Результаты всех запросов типа 0 записываются в output.txt.
Сложность - $O(\log n)$ на каждую операцию.

Описание проведённых тестов

input.txt			
1	11		
2	+1 5		
3	+1 3		
4	+1 7		
5	0 1		
6	0 2		
7	0 3		
8	-1 5		
9	+1 10		
10	0 1		
11	0 2		
12	0 3		
13			

output.txt	
1	7
2	5
3	3
4	10
5	7
6	3

```

7 ▶ def test_minimal():  ⚡ MarieeMorte
8     check( task_num: 16, input_data: "2\n+1 5\n0 1\n", expected: "5")
9
10
11 ▶ def test_example():  ⚡ MarieeMorte
12     check(
13         task_num: 16,
14         input_data: "11\n"
15             "+1 5\n"
16             "+1 3\n"
17             "+1 7\n"
18             "0 1\n"
19             "0 2\n"
20             "0 3\n"
21             "-1 5\n"
22             "+1 10\n"
23             "0 1\n"
24             "0 2\n"
25             "0 3\n",
26         expected: "7\n5\n3\n10\n7\n3",
27     )

30 ▶ def test_maximum():  ⚡ MarieeMorte
31     n = 100_000
32     half = n // 2
33     lines = [str(n)]
34     for i in range(1, half + 1):
35         lines.append(f"+1 {i}")
36     expected = []
37     for k in range(1, half + 1):
38         lines.append(f"0 {k}")
39         expected.append(str(half - k + 1))
40     input_data = "\n".join(lines) + "\n"
41     check( task_num: 16, input_data, "\n".join(expected), timeout=60)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.083s	memory=272.0 KB
Пример из задачи	time=0.227s	memory=328.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.969s	memory=14.8 MB

Вывод по задаче

Реализована структура данных на основе AVL-дерева с поддержкой порядковых статистик. Хранение размеров поддеревьев позволяет находить k -й максимум за логарифмическое время, а балансировка гарантирует эффективность при 10^5 операций.

Вывод по проделанной работе

В ходе выполнения лабораторной работы были изучены и реализованы двоичные деревья поиска, AVL-деревья и Splay-деревья, а также алгоритмы их обхода и валидации. Освоены итеративные обходы без рекурсии, балансировка через повороты, порядковые статистики и амортизационный анализ. Все задачи решены на языке Python и укладываются в заданные ограничения по времени и памяти.